Angular HandsOn 2: Components and Data Binding

HANDS-ON 2 [Beginner]

Data Binding, Lifecycle Hooks & Component Communication

Topics Covered:

Property Binding[Check] Event Binding[Check]

Two-Way Binding (ngModel)[Check] Lifecycle Hooks ? ngOnInit, [Check]

ngOnChanges, ngOnDestroy

@Input and @Output Decorators[Check] EventEmitter[Check]

You will now make the Student Course Portal dynamic using Angular's four binding types, implement lifecycle hooks t- control component behaviour, and establish parent-child communication using @Input and @Output.

Task 1: All Four Binding Types

Goal: Practise interpolation, property binding, event binding, and two-way binding in one component.

Steps:

11. In HomeComponent, declare a TypeScript property: portalName = 'Student Course Portal'. Use string interpolation in the template t- display it: <h1>{{ portalName }}</h1>.

12. Add a property isPortalActive = true. Use property binding t- control a button's disabled state: <button [disabled]='!isPortalActive'>Enroll Now</button>.

13. Add a method onEnrollClick() that sets a property message = 'Enrollment opened!'. Use event binding on the button: (click)='onEnrollClick()'.

14. Add an <input> for a search term and a property searchTerm = ''. Use two-way binding with ngModel: <input [(ngModel)]='searchTerm'>. Import FormsModule in the module/component. Display the live value below the input: Searching for: {{ searchTerm }} .

15. Explain in a code comment the difference between [property] (one-way, component ? DOM) and [(ngModel)] (two-way, DOM ? component).

Hint:

- Two-way binding [(ngModel)] is shorthand for [ngModel]='prop' (ngModelChange)='prop=$event' ? you can always split it int- one-way bindings if you need t- intercept the change.
- FormsModule must be imported in the component's imports array (standalone) or the module's imports (NgModule) for ngModel t- work.

Expected Outcome: Typing in the search box updates the 'Searching for:' text in real time. Clicking Enroll Now

shows the message. Button is disabled/enabled based on isPortalActive.

Task 2: Lifecycle Hooks

Goal: Implement the three most important lifecycle hooks and observe when they fire.

Steps:

16. In HomeComponent, implement ngOnInit: fetch (or simulate) a count of available courses and log

'HomeComponent initialised ? courses loaded' t- the console.
17. Implement ngOnDestroy: log 'HomeComponent destroyed' t- the console. Navigate away from the home page and back ? observe both logs in the browser console.
18. Create a new CourseCardComponent: ng generate component components/course-card. Add an @Input() course: any property. Implement ngOnChanges(changes: SimpleChanges): log the previous and current value of the course input whenever it changes.
19. In CourseListComponent, render three CourseCardComponents with different hardcoded course inputs. Open the console and verify ngOnChanges fires once per card on initial render.
Hint:

- ngOnInit fires once after the component's inputs are first set ? use it for data fetching, not the constructor (which fires before inputs are set).
- ngOnDestroy is critical for unsubscribing from Observables and clearing timers ? forgetting it causes memory leaks in long-running SPAs.
Expected Outcome: Console shows ngOnInit log on Home page load, ngOnDestroy log on navigation away, and ngOnChanges logs with previous/current values for each card.

Task 3: @Input and @Output ? Parent-Child Communication

Goal: Pass data down with @Input and emit events up with @Output.
Steps:
20. In CourseCardComponent, add @Input() course: { id: number, name: string, code: string, credits: number } and render all four fields in the template.
21. Add an @Output() enrollRequested = new EventEmitter<number>() t- CourseCardComponent. Add an 'Enroll' button in the template: <button (click)='enrollRequested.emit(course.id)'>Enroll</button>.
22. In CourseListComponent, define a courses array with 5 course objects. Render a CourseCardComponent for each: <app-course-card *ngFor='let c of courses' [course]='c' (enrollRequested)='onEnroll($event)'></app-course-card>.
23. Implement onEnroll(courseId: number) in CourseListComponent: log 'Enrolling in course: ' + courseId and update a selectedCourseId property.
24. Display the selectedCourseId below the list: <p *ngIf='selectedCourseId'>Selected course ID: {{ selectedCourseId }}</p>.
Hint:
- @Input and @Output create a one-way data flow: data flows down via @Input, events bubble up via @Output. This is Angular's recommended pattern for parent-child communication.
- @Output with EventEmitter<T> where T is the payload type ? using the generic type makes the event strongly typed.

Expected Outcome: Clicking Enroll on any card logs the course ID t- the console and shows the selected ID below the list.